



CARMA

Senior Fullstack Engineer

Home Assignment

Media Signal Service

Build a small fullstack service that ingests news articles, enriches them with AI, and lets users search and explore the data. The project mirrors real challenges in media intelligence: boolean query parsing, LLM-driven enrichment, SQL performance, and responsible handling of untrusted content.

Ground Rules

- **AI tools are expected.** Use any coding assistant, LLM, or tool you like — this is the job. What we evaluate is your judgment, not whether you typed every character.
- **Don't gold-plate.** We grade decisions and tradeoffs, not completeness or pixel-perfect design. A well-reasoned "I stubbed this and here's how I'd finish it" scores better than a rushed, brittle implementation.
- **Estimated effort: ~7–9 focused hours** (with AI assistance). Per-section estimates are listed below. Return your submission within **5 calendar days** of receiving this brief.
- **Tech stack:** Ideally use our preferred tech stack but if not comfortable with them yet, you can use whatever you're most productive in (Node / React, PostgreSQL, etc.). Justify your choices in your README.

What to Build

1. Data Layer & SQL (~30 min)

Design a database schema and load the provided sample_articles.json (20 articles with headlines, bodies, sources, dates, and languages).

Requirements:

- The schema must support efficient filtering by date range, source, and language.
- Implement a paginated article list endpoint that avoids the performance pitfalls of deep OFFSET pagination.
- Support an aggregate endpoint: article counts grouped by month (or week) for a given filter set, suitable for powering a dashboard chart.
- Include appropriate indexes and briefly explain your indexing choices in your README.

2. Boolean Search (~90–120 min)

Implement a search feature that filters articles using boolean query syntax:

- **AND**, **OR**, **AND NOT** operators
- Nested brackets: (a AND (b OR c))
- Phrase matching with double quotes: "media intelligence"
- Wildcard with *: renew* matches "renewable", "renewables", etc.
- AND, OR, and AND NOT are case-sensitive keywords (lowercase "and" should be treated as a search term, not an operator)

Approach is fully open. You may hand-roll a parser, use PostgreSQL full-text search (to_tsquery / websearch_to_tsquery), use a library, or combine approaches. Explain your choice and its tradeoffs in your README.

Example queries the system should handle:

```
"oil prices" AND (geopolitical OR "supply chain")
```

```
renewable AND NOT (nuclear OR coal)
```

```
AI AND ("healthcare" OR "diagnostic") AND NOT startup*
```

Query

```
Carma AND ( "Media Monitoring" OR "Media Analysis" OR "Media Intelligence" ) AND NOT ( "Social  
Networking" OR Gaming OR Karma* )
```

3. LLM Enrichment (~60–90 min)

For each article, use an LLM to produce:

- A 1–2 sentence summary
- A sentiment classification (positive / negative / neutral / mixed)
- 1–3 topic tags (e.g. “Energy”, “Geopolitics”, “Technology”)

Requirements:

- Choose your model(s) and justify the choice on cost, quality, and latency. If you would use different models for different tasks (e.g. a cheaper model for classification, a stronger one for summarisation), explain why.
- Include a cost estimate: what would this cost at 50,000 articles/day?
- Implement at least one cost guardrail (e.g. token budget per article, rate limiting, input stripping, caching).

You may use any LLM API (OpenAI, Anthropic, local models, etc.) or mock the LLM responses if you prefer not to incur API costs — but if you mock, include realistic example outputs and document what the real integration would look like.

4. Fullstack Surface (~90–120 min)

Build a minimal API + UI that demonstrates the service:

- **Search page:** text input for boolean queries, displays matching articles with their AI-generated summary, sentiment badge, and topic tags.
- **Aggregate view:** a simple chart or table showing article counts over time (by month or week), filterable by at least one dimension (e.g. sentiment, source, or topic).
- The UI does not need to be beautiful — functional and clear is sufficient.

5. Responsible Coding (~45–60 min)

Demonstrate awareness of production-grade concerns:

- **SQL injection prevention:** all queries use parameterised inputs or whitelisted identifiers (no string interpolation of user input into SQL).
- **XSS prevention:** article bodies in the sample data contain deliberate HTML/script injection attempts. Your UI must handle these safely.
- **Prompt injection awareness:** article text is untrusted user content being fed to an LLM. Document (and ideally implement) at least one mitigation.
- **Cost / rate guardrails:** prevent runaway LLM costs (e.g. per-request token cap, daily budget limit, duplicate detection).

You do not need to implement authentication, HTTPS, or deployment infrastructure.

What to Submit

A git repository (GitHub, GitLab, or a zip) containing:

1. Working Code

The service should be runnable with a simple setup (ideally docker-compose up or a few shell commands documented in the README).

2. README with Required Sections

Section	What to Include
Plan	Your plan before you started coding: how you broke down the problem, what you tackled first and why, what you would cut if time ran short.
Architecture & Decisions	Tech stack choices, schema design, boolean search approach, LLM model selection with cost / quality / latency tradeoffs.
LLM Cost Analysis	Model(s) chosen, per-article cost breakdown, projected daily cost at 50K articles, guardrails implemented.
Security & Responsibility	How you handled SQL injection, XSS, prompt injection, and cost controls.
Reflection	Where AI helped you most, where it misled you or produced something you had to fix, and what you would do differently with more time.

3. LLM Transcript

Include the full transcript (or exported chat history) of your interactions with AI tools during this assignment. We use this to evaluate:

- How you framed problems for the AI
- Whether you critically evaluated and refined AI output
- How you handled cases where the AI was wrong or incomplete

This is not about catching you using AI — we expect it. It's about seeing how you work with AI: do you drive it with clear intent and verify the output, or do you accept everything uncritically?

Evaluation Criteria

Criterion	Weight	What We Look For
Plan quality	15%	Clear problem decomposition, realistic prioritisation, awareness of what to cut under time pressure.
Correctness & functionality	20%	Does the search work? Does the enrichment produce reasonable results? Does the aggregate query return correct data?
SQL & performance	15%	Sensible schema, appropriate indexes, avoidance of common pitfalls (N+1, deep OFFSET, SELECT *).
LLM selection & cost reasoning	20%	Thoughtful model choice with clear tradeoff analysis, realistic cost estimate, implemented guardrails.
Security & responsibility	15%	SQL injection prevention, XSS handling, prompt injection awareness, cost controls.
Transcript & reflection	15%	Evidence of critical thinking in AI interactions, honest reflection on what worked and what didn't.

Sample Data

The file **sample_articles.json** is provided alongside this brief. It contains 20 articles with deliberate edge cases:

- Articles 6 and 18 contain HTML/script injection in headlines or bodies (for XSS testing)
- Articles 8 and 9 are in Arabic and Chinese respectively (for multilingual handling)
- Article 17 has an empty headline (for null / edge-case handling)
- Sources, dates, and languages are varied to test filtering and aggregation

Good luck. We look forward to seeing how you think.